Материалът за петнадесетата лекция от записките:

глава 4.4 (**Кодиране на машина на Тюринг. Диагоналният език. Универсална машина на Тюринг**), стр. 183-186,

глава 4.9 (**Сложност**), стр. 206.

За допълнително четене от учебника на Hopcroft, Motwani, Ullman:

глава 9.1 (A Language That Is Not Recursively Enumerable), глава 9.2 (An Undecidable Problem That is RE), глава 10.1 (The Classes $\mathcal{P}$ and $\mathcal{NP}$), глава 10.2 (An $\mathcal{NP}$-Complete Problem).

Започнахме лекцията с кодиране на машина на Тюринг. Целта беше да съпоставим по изчислим начин на всяка машина на Тюринг $\mathcal{M}$ дума от $\{0,1\}^\star$, която да съдържа цялата информация за машината. Първо показахме как се кодира един преход, след това в кода на $\mathcal{M}$ подредихме кодовете на всички преходи на $\mathcal{M}$ с подходящи разделители между тях (подредени спрямо каноничната наредба). Също добавихме информация за броя на буквите в азбуката Σ.

Формалното описание на кодирането е на стр. 184 в записките.

От всяка машина $\mathcal{M}$ може да се получи ефективно нейният код $\ulcorner\mathcal{M}\urcorner$. Обратно, от дадена дума $\omega \in \{0,1\}^\star$ може да се получи ефективно машината $\mathcal{M}_\omega$ с код ω. Разбира се, не всяка дума ω е код на машина на Тюринг, но езикът $L_{code} \subset \{0,1\}^\star$ от всички кодове на машини на Тюринг е разрешим.

За да можем да работим с една на машина на Тюринг на практика няма разлика дали е дадена самата машина или е даден нейният код. От друга страна, кодирането дава възможност да използваме машини на Тюринг като данни в рамките на изчислението на друга машина на Тюринг (в частност да извършваме симулация на кодираните машини).

Нека $L_{code} = \{\omega_1,\ \omega_2,\ \ldots,\ \omega_n,\ \ldots\}$. Стана ясно, че списъкът

$$\mathcal{M}_{\omega_1},\ \mathcal{M}_{\omega_2},\ \ldots,\ \mathcal{M}_{\omega_n},\ \ldots$$

съдържа всички машини на Тюринг. Да разгледаме следната безкрайна таблица:

	$\mathcal{M}_{\omega_1}$	$\mathcal{M}_{\omega_2}$	$\ldots$	$\mathcal{M}_{\omega_n}$	$\ldots$
ω_1	$\boxed{0}$	1	$\ldots$	0	$\ldots$
ω_2	1	$\boxed{1}$	$\ldots$	0	$\ldots$
$\vdots$	$\vdots$	$\vdots$	$\ldots$	$\vdots$	$\ldots$
ω_n	0	1	$\ldots$	$\boxed{0}$	$\ldots$
$\vdots$	$\vdots$	$\vdots$	$\ldots$	$\vdots$	$\ldots$

На ред i и стълб j сме поставили 1, ако $\omega_i \in L(\mathcal{M}_{\omega_j})$ и 0, ако $\omega_i \notin L(\mathcal{M}_{\omega_j})$.

Тъй като всеки полуразрешим език се разпознава от някоя машина на Тюринг, по стълбовете на таблицата получаваме всички полуразрешими езици, които се състоят от думи от L_{code}. Нашата непосредствена цел ще бъде да конструираме език L_{diag}, който не е полуразрешим, затова ще се погрижим той да се различава в поне една позиция от всеки стълб в таблицата.

И така на позиции $1,1$ имаме 0, затова ще вземем $\omega_1 \in L_{diag}$. Този избор гарантира, че $L_{diag} \neq L(\mathcal{M}_{\omega_1})$, тъй като $\omega_1 \notin L(\mathcal{M}_{\omega_1})$.

На позиции $2,2$ имаме 1, затова ще вземем $\omega_2 \notin L_{diag}$ и така гарантираме $L_{diag} \neq L(\mathcal{M}_{\omega_2})$.

По същия начин постъпваме с всички останали диагонални позиции: на n,n имаме 0 и ще вземем $\omega_n \in L_{diag}$ и т.н.

Получаваме езикът L_{diag}, който е различен от всички езици $L(\mathcal{M}_{\omega_n})$, така че той не е полуразрешим.

През първия семестър видяхме аналогично разсъждение (диагонален метод на Кантор), с което показахме, че множествата от естествени числа не могат да се подредят в безкрайна редица. По-точно, за всяка редица от множества от естествени числа построихме ново множество от естествени числа, което не е член на редицата.

Теорема 1 *Диагоналният език*

$$L_{diag} \;=\; \{\, \omega \in \{0,1\}^\star \mid \omega \in L_{code} \;\&\; \omega \notin L(\mathcal{M}_\omega) \,\}$$

не е полуразрешим, т.е. не се разпознава от машина на Тюринг.

Доказателство. Да допуснем, че $L_{diag} = L(\mathcal{M})$. Нека $\omega = \ulcorner \mathcal{M} \urcorner$ е кодът на $\mathcal{M}$. Тогава имаме $\omega \in L_{diag} \iff \omega \notin L(\mathcal{M}_\omega) \iff \omega \notin L(\mathcal{M})$. Достигнахме до противоречие с $L_{diag} = L(\mathcal{M})$ (по-точно, никое съждение не е логически еквивалентно на своето отрицание).

Теорема 2 *Езикът*

$$L_{accept} \;=\; \{\, \omega \in \{0,1\}^\star \mid \omega \in L_{code} \;\&\; \omega \in L(\mathcal{M}_\omega) \,\}$$

е полуразрешим, но не е разрешим.

Доказателство. Не е възможно L_{accept} да е разрешим, тъй като $L_{diag} = L_{code} \setminus L_{accept}$ и разрешимите езици са затворени относно всички булеви операции. За да покажем, че L_{accept} е полуразрешим, при вход ω извършваме симулация на машината $\mathcal{M}_\omega$ над вход ω. Симулацията завършва точно когато $\omega \in L_{accept}$. Повече подробности има на страница 185 в записките.

Като следствие от теоремата на Пост получаваме, че допълнението $\overline{L_{accept}}$ не е полуразрешимо.

Теорема 3 (неразрешимост на стоп-проблема) *Езикът*

$$L_{halt} \;=\; \{\, \ulcorner \mathcal{M} \urcorner \sharp \omega \mid \textit{машината } \mathcal{M} \textit{ спира над } \omega \,\}$$

е полуразрешим, но не е разрешим.

Полуразрешимостта на L_{halt} получаваме както в предната теорема, в случая със симулация на $\mathcal{M}$ над ω. Симулацията винаги считаме за успешна (независимо от последното състояние) стига да завърши.

Допускаме, че L_{halt} е разрешим и нека $\mathcal{M}_{halt}$ е разрешител за L_{halt}.

Ще построим разрешител $\mathcal{M}'$ за L_{accept}.

При даден вход ω:

1. $\mathcal{M}'$ проверява дали $\omega \in L_{code}$ и ако не е вярно, $\mathcal{M}'$ отхвърля ω.

2. Ако $\omega \in L_{code}$, $\mathcal{M}'$ симулира работата на $\mathcal{M}_{halt}$ върху $\omega\sharp\omega$.

3. Ако симулацията завърши в q_{reject}, $\mathcal{M}'$ отхвърля ω.

4. Ако симулацията завърши в q_{accept}, $\mathcal{M}'$ симулира $\mathcal{M}_\omega$ над вход ω.

5. Ако последната симулация завърши успешно, $\mathcal{M}'$ приема ω, ако завърши неуспешно, $\mathcal{M}'$ отхвърля ω.

Получихме, че $L(\mathcal{M}') = L_{accept}$ и $\mathcal{M}'$ е разрешител, което е противоречие с факта, че L_{accept} е неразрешим.

Идеята на доказателството е, че сведохме L_{accept} към L_{halt}. Успяхме да направим симулация на $\mathcal{M}_\omega$ над вход ω, която винаги завършва, тъй като чрез $\mathcal{M}_{halt}$ премахнахме случая, в който симулацията не завършва.

Теорема 4 *Универсалният език*

$$L_{univ} \;=\; \{\, \ulcorner\mathcal{M}\urcorner\sharp\omega \mid \mathcal{M} \text{ е машина на Тюринг и } \omega \in L(\mathcal{M}) \,\}$$

е полуразрешим, но не е разрешим.

Доказателство. Строим многолентова машина $\mathcal{U}$, която при вход α:

1. Проверява дали α има вида $\ulcorner\mathcal{M}\urcorner\sharp\omega$ и ако не, $\mathcal{U}$ отхвърля α.

2. Ако $\alpha = \ulcorner\mathcal{M}\urcorner\sharp\omega$, то $\mathcal{U}$ симулира работата на $\mathcal{M}$ върху вход ω. Симулацията ще се извършва на отделна лента, на която първоначално копираме ω. Ще има допълнителна лента, върху която се пази текущото състояние на $\mathcal{M}$. На всяка стъпка $\mathcal{U}$ претърсва кода на $\mathcal{M}$ за необходимия преход от текущото състояние и текущата буква на $\mathcal{M}$. Като го намери, записва новото състояние на $\mathcal{M}$ и променя лентата за симулация по съответен начин.

3. Тъй като $\mathcal{U}$ напълно копира поведението на $\mathcal{M}$:

$\mathcal{U}$ приема α точно когато $\mathcal{M}$ приема ω,

$\mathcal{U}$ отхвърля α точно когато $\mathcal{M}$ отхвърля ω,

$\mathcal{U}$ зацикля над α точно когато $\mathcal{M}$ зацикля над ω.

По този начин, $L(\mathcal{U}) = L_{univ}$. Естествено е да наречем $\mathcal{U}$ *универсална машина на Тюринг*, тъй като това е една единствена машина на Тюринг, която може да симулира изчислението на всички машини на Тюринг. Счита се, че това е формалният първообраз на съвременните компютри и по-точно на идеята, че съдържанието на програмите трябва да се съхранява като вид данни в паметта.

Дефиниция 5 *Нека $K, L \subseteq \Sigma^\star$ и $f : \Sigma^\star \to \Sigma^\star$. Казваме, че функцията f свежда K към L, ако:*

$$\forall\omega \in \Sigma^\star \;:\; \omega \in K \iff f(\omega) \in L.$$

Ще отбелязваме, че $K \leq_m L$, ако съществува изчислима f, която свежда K към L.

Лесно се съобразяват следните две свойства:

1. $K \leq_m L$ и L е разрешим влече, че K е разрешим.
2. $K \leq_m L$ и L е полуразрешим влече, че K е полуразрешим.

Например, $L_{accept} \leq_m L_{univ}$ посредством $f(\omega) = \omega\sharp\omega$. Тъй като L_{accept} не е разрешим, L_{univ} също не е разрешим.

Сложност

Машината на Тюринг $\mathcal{M}$ *работи за полиномиално време*, ако съществува полином $p(x) \in \mathbb{N}[x]$, такъв че

$$\forall \omega \in \Sigma^\star \ : \ (\epsilon, q_{start}, \omega\sqcup) \ \vdash_{\mathcal{M}}^{p(|\omega|)+1} \ (\lambda, q, \rho),$$

където $\lambda, \rho \in \Gamma^\star$ и $q \in Q$. С други думи, за произволен вход ω, дължината на произволен клон в дървото изчислението не може да надвишава $p(|\omega|)$. Очевидно е, че $\mathcal{M}$ е разрешител.

Означаваме:

$$\begin{aligned}
\mathcal{P} \ &= \ \{ \ L \subseteq \Sigma^\star \mid L = L(\mathcal{M}) \text{ за някоя детерминирана } \mathcal{M}, \\
&\qquad\qquad \text{която работи за полиномиално време } \}, \\
\mathcal{NP} \ &= \ \{ \ L \subseteq \Sigma^\star \mid L = L(\mathcal{M}) \text{ за някоя недетерминирана } \mathcal{M}, \\
&\qquad\qquad \text{която работи за полиномиално време } \}.
\end{aligned}$$

Машината на Тюринг $\mathcal{M}$ *работи за експоненциално време*, ако съществува полином $p(x) \in \mathbb{N}[x]$, такъв че

$$\forall \omega \in \Sigma^\star \ : \ (\epsilon, q_{start}, \omega\sqcup) \ \vdash_{\mathcal{M}}^{2^{p(|\omega|)}+1} \ (\lambda, q, \rho),$$

където $\lambda, \rho \in \Gamma^\star$ и $q \in Q$. Нека

$$\begin{aligned}
\mathcal{EXP} \ &= \ \{ \ L \subseteq \Sigma^\star \mid L = L(\mathcal{M}) \text{ за някоя детерминирана } \mathcal{M}, \\
&\qquad\qquad \text{която работи за експоненциално време } \}.
\end{aligned}$$

Знаем за следните включвания:

$$\text{крайни} \subseteq \text{регулярни} \subseteq \text{безконтекстни} \subseteq \mathcal{P} \subseteq \mathcal{NP} \subseteq \mathcal{EXP}.$$

Първите две включвания са строги. Езикът $\{a^n b^n c^n \mid n \in \mathbb{N}\}$ очевидно е в $\mathcal{P}$ (вече построихме подходяща машина), така че третото включване също е строго. Комбинацията на построяването на нормална форма на Чомски с CYK-алгоритъма може да се симулира за полиномиално време. Включването $\mathcal{NP} \subseteq \mathcal{EXP}$ следва от теоремата за симулация на недетерминирана машина с детерминирана. Забележете, че клоните на изчислението имат

полиномиална дължина, но броят на тези клони е експоненциален и в симулацията всичките трябва да бъдат обходени (в най-лошия случай).

За последните две включвания въпросът дали са строги е отворен. Безспорно $\mathcal{P} = \mathcal{NP}$? е най-популярният отворен проблем в теоретичната информатика.

Теорема 6 *Езикът L принадлежи на $\mathcal{NP}$ тогава и само тогава, когато съществуват език $V \in \mathcal{P}$ и полином $q(x) \in \mathbb{N}[x]$, такива че*

$$\forall \omega \in \Sigma^\star \ : \ \omega \in L \iff \exists \alpha, \ |\alpha| \leq q(|\omega|) \ \ \& \ \ \omega \sharp \alpha \in V.$$

Доказателство. Естествено е да наречем V *валидатор* и α да е *свидетел* за ω. По този начин, една дума $\omega \in L$ точно когато за нея съществува свидетел α с подходяща дължина, който може да се валидира с V.

За посоката ($\Longrightarrow$). Нека $\mathcal{N}$ е недетерминирана машина, която работи за полиномиално време $p(x)$ и $L(\mathcal{N}) = L$. Избираме $q(x) = p(x)$. За да опишем валидатора V използваме машината $\mathcal{D}$ от теоремата за детерминизация. Строи следната машина $\mathcal{M}$: при вход $\omega \sharp \alpha$,

1. $\mathcal{M}$ копира ω върху лентата за симулация на $\mathcal{D}$;

2. $\mathcal{M}$ копира α върху управляващата лента на $\mathcal{D}$;

3. $\mathcal{M}$ извършва една итерация от цикъла на изчислението на $\mathcal{D}$, т.е. симулира машината $\mathcal{N}$ върху вход ω съобразно съдържанието α на управляващата лента.

4. Ако симулацията завърши в q_{accept}, то $\mathcal{M}$ приема $\omega \sharp \alpha$. В противен случай, $\mathcal{M}$ отхвърля $\omega \sharp \alpha$ (напомням, това означава, че симулацията завършва в друго състояние или се стига до бленк или до символ, за който няма съответен преход върху управляващата лента).

Нека $V = L(\mathcal{M})$. Ясно е, че $V \in \mathcal{P}$ (с линеен полином).

Нека $\omega \in L$. Тогава $(\epsilon, q_{start}, \omega \sqcup) \ \vdash_{\mathcal{N}}^{\leq q(|\omega|)} \ (\lambda, q_{accept}, \rho)$. Нека α кодира това изчисление (съобразно доказателството от теоремата за детерминизация на машина на Тюринг). Тогава $|\alpha| \leq q(|\omega|)$ и от построението на $\mathcal{M}$ е ясно, че $\omega \sharp \alpha \in V$. Обратно, ако $\omega \sharp \alpha \in V$, то $\omega \in L$, тъй като това е единственият случай, при който $\mathcal{M}$ завършва успешно.

За посоката ($\Longleftarrow$). Да изберем полином $q(x) \in \mathbb{N}[x]$ и език V с описаните свойства, $V = L(\mathcal{M})$ и $\mathcal{M}$ работи за полиномиално време $p(x)$. Строи недетерминирана машина $\mathcal{N}$: при вход $\omega \in \Sigma^\star$,

1. Избира недетерминистично α с дължина $|\alpha| \leq q(|\omega|)$ (има преходи, гарантиращи че от подходяща конфигурация $(\lambda, q, \sqcup\rho)$ може за една стъпка да се стигне до $(\lambda, q, x\rho)$ за всяка стойност на x).

2. Симулира $\mathcal{M}$ върху вход $\omega \sharp \alpha$.

Всеки клон на изчислението има дължина $\leq q(|\omega|) + p(|\omega| + 1 + q(|\omega|))$, което е полином на $|\omega|$. Така $\mathcal{N}$ работи за полиномиално време и също е ясно, че $L(\mathcal{N}) = L$, тъй като $\omega \in L(\mathcal{N})$ точно когато в изчислението има поне един успешен клон, което е изпълнено точно когато има свидетел с описаните свойства, т.е. когато $\omega \in L$.

Дефиниция 7 *За езици $K, L \in \Sigma^\star$, бележим $K \leq_m^P L$ ако $K \leq_m L$ и свеждащата функция f е изчислима с детерминистична машина на Тюринг за полиномиално време.*

Лесно се съобразява, че:
1. Ако $L \in \mathcal{P}$ и $K \leq_m^P L$, то $K \in \mathcal{P}$.
2. Ако $L \in \mathcal{NP}$ и $K \leq_m^P L$, то $K \in \mathcal{NP}$.

Дефиниция 8 *Един език $L \in \mathcal{NP}$ е $\mathcal{NP}$-пълен, ако за всеки $K \in \mathcal{NP}$ имаме $K \leq_m^P L$.*

Нека SAT е езикът, който се състои от булевите функции f в конюнктивна нормална форма, които приемат поне една стойност истина.

Теорема 9 (Кук-Левин) *Езикът SAT е $\mathcal{NP}$-пълен.*

Други $\mathcal{NP}$-пълни езици са: (1) графите, в които има Хамилтонов цикъл; (2) редиците от естествени числа $a_1, \ldots, a_n, W$, такива че W е сума на някои от a-тата. Съществуват още стотици примери.

$\mathcal{NP}$-пълните езици са най-сложните езици в класа $\mathcal{NP}$. Всеки два от тях се свеждат полиномиално един към друг. Важността им по отношение на проблема $\mathcal{P} = \mathcal{NP}$? произтича от следните еквивалентности:

$$\mathcal{P} = \mathcal{NP} \iff \text{за някой } \mathcal{NP}\text{-пълен език } L,\ L \in \mathcal{P},$$

$$\mathcal{P} \neq \mathcal{NP} \iff \text{за някой } \mathcal{NP}\text{-пълен език } L,\ L \notin \mathcal{P}.$$